UNIVERSIDAD DE EL SALVADOR
Facultad Multidisciplinaria De Occidente
Departamento de Ingeniería y Arquitectura
Ingeniería en Desarrollo de Software

Asignatura:

DESARROLLO Y REUTILIZACIÓN DE SOFTWARE

Ciclo X / Quinto año

Tema:

Proyecto

Tema 2: Aplicación de E-commerce utilizando el Patrón MVC

Tutor GT02:

Ing. Dinora Virginia de Melara

Presentado por:

Fernando José Rosales Valdés	**RV19012**
David Salomón Martínez Valladares	**MV12013**
Milton Obed Alas Hernández	**AH09062**
José Mauricio Chavarría González	**CG92088**
Jose Franklin Angel Guevara	**AG19030**
Alexis Alfonso Alvarenga Alvarenga	**AA22044**

Fecha:

25 de octubre de 2026

Informe del Proyecto

Introducción

En este proyecto desarrollamos una tienda en línea (e-commerce) sencilla. En la aplicación el usuario puede ver un catálogo de productos, agregarlos a un carrito, quitar productos del carrito y finalizar la compra. Cada compra queda guardada como una orden y se puede consultar en un historial.

Para organizar el código usamos el patrón Modelo-Vista-Controlador (MVC). Este patrón separa la aplicación en tres partes, cada una con una tarea distinta, lo que hace que el código sea más ordenado, fácil de entender y fácil de modificar.

Objetivo

Aplicar los conceptos de reutilización de software construyendo una aplicación web modular con el patrón MVC, en la que los productos, las ordenes de compras y sus detalles esten separados en componentes que se puedan entender y reutilizar por separado.

Tecnologías utilizadas

Las indicaciones sugerían C# con ASP.NET Core, pero la tutora nos autorizó a usar el lenguaje de nuestra preferencia. Elegimos Java con Spring Boot porque funciona de la misma forma que ASP.NET Core MVC: también tiene controladores, rutas y vistas.

- **Java 17:** lenguaje de programación.
- **Spring Boot:** framework para crear la aplicación web, las rutas y los controladores.
- **Thymeleaf:** motor de plantillas para crear las vistas en HTML.
- **Spring Data JPA:** para guardar y leer datos sin escribir consultas SQL a mano.
- **H2:** base de datos en memoria, que no necesita instalación.
- **Bootstrap 5:** estilos para que las páginas se vean ordenadas.
- **Maven:** para manejar las librerías y ejecutar el proyecto.

Enfoque utilizado: el patrón MVC

Dividimos la aplicación en las tres capas del patrón. Cada carpeta del proyecto corresponde a una parte del patrón.

Modelo. Representa los datos y las reglas del negocio. Está formado por:

- **Producto:** nombre, descripción, precio y stock de cada artículo.
- **Orden:** la compra realizada, su fecha y su total.
- **DetalleOrden:** cada producto de una orden, con su cantidad y el precio al momento de la compra.
- **Carrito e ItemCarrito:** los productos que el usuario va eligiendo antes de comprar.
- **Repositorios** (ProductoRepository, OrdenRepository): se encargan de guardar y buscar datos en la base de datos.
- Servicios (ProductoService): contiene la lógica de negocio para gestionar los productos. La conversión del carrito en una orden se maneja en el controlador utilizando el repositorio de órdenes.

Vista. Es lo que el usuario ve en el navegador. Son páginas HTML hechas con Thymeleaf:

- **productos/lista.html:** catálogo de productos.
- **carrito/detalle.html:** carrito con subtotales y total a pagar.

Controlador. Recibe lo que hace el usuario (por ejemplo, un clic en un botón), le pide al modelo lo necesario y decide qué vista mostrar:

- **ProductoController:** muestra el catálogo.
- **CarritoController:** agrega y quita productos, y procesa la compra.

Decisiones tomadas

- **Usar Spring Boot en lugar de ASP.NET Core:** con la autorización de la tutora, porque el grupo tiene más experiencia en Java y Spring Boot aplica el patrón MVC de la misma manera.
- **Base de datos H2 en memoria:** no hay que instalar nada para probar el proyecto. Al iniciar, la aplicación carga cuatro productos de prueba automáticamente.

- **Carrito guardado en la sesión del usuario:** cada persona tiene su propio carrito. El carrito no se guarda en la base de datos; solo la compra final se guarda como orden.
- **Guardar el precio en cada detalle de la orden:** así, si un producto cambia de precio después, las órdenes anteriores no cambian.
- **Usar BigDecimal para los precios de las órdenes:** evita errores de redondeo con el dinero.
- **Inyección de dependencias por constructor:** cada clase recibe lo que necesita (por ejemplo, un servicio recibe su repositorio), así las clases no dependen de cómo se crean las otras.

Reutilización del código

Aplicamos la reutilización de varias formas:

- **Repositorios de Spring Data JPA:** al heredar de JpaRepository, cada repositorio obtiene métodos ya hechos para guardar, buscar, listar y eliminar, sin escribir ese código. .
- **Servicios compartidos:** la lógica está en los servicios y no en los controladores, por eso un mismo servicio lo usan varios controladores. Por ejemplo, ProductoService lo usan ProductoController y CarritoController.
- **Clases con una sola responsabilidad:** la clase Carrito sabe agregar productos, quitarlos y calcular el total. Los controladores solo la usan y no repiten esa lógica.
- **Misma estructura para cada entidad:** Orden y detalle orden siguen el mismo esquema que Producto (modelo, repositorio y servicio). Para agregar una nueva entidad solo hay que repetir ese esquema.
- **Componentes de terceros:** reutilizamos Spring Boot, Thymeleaf y Bootstrap en lugar de programar desde cero el servidor web, las plantillas y los estilos.

Cómo interactúan las partes del sistema

Este es el recorrido de una compra completa, paso a paso:

● Ver el catálogo: El usuario entra a la tienda. ProductoController le pide la lista de productos a ProductoService, que la obtiene de la base de datos usando ProductoRepository. Luego, se muestra la vista del catálogo (lista.html).

● Agregar al carrito: El usuario hace clic en "Agregar al carrito". CarritoController busca el producto usando ProductoService y lo agrega al carrito que se guarda en la sesión del usuario (HttpSession). El sistema lo redirige de vuelta al catálogo, donde el contador del carrito se actualiza en tiempo real.

● Ver el carrito: El usuario hace clic en "Ver carrito". CarritoController recupera el carrito de la sesión y muestra la vista detalle.html con los productos, sus cantidades, subtotales y el total a pagar.

● Procesar la compra: Al hacer clic en "Procesar compra", CarritoController toma los productos del carrito y crea una nueva Orden. Luego, por cada producto, genera un DetalleOrden (guardando la cantidad y el precio histórico). Finalmente, guarda todo en la base de datos usando OrdenRepository.

● Finalizar: Se vacía el carrito de la sesión y el usuario vuelve al catálogo con el mensaje "¡Compra realizada con éxito!". La compra ya quedó registrada de forma permanente en la base de datos.

● Verificación (Base de datos): Si se ingresa a la consola de H2, se puede ejecutar SELECT * FROM ORDENES y SELECT * FROM DETALLE_ORDEN para comprobar que la compra y sus productos se guardaron correctamente.

En todos los pasos se cumple la misma regla: la vista nunca habla directamente con la base de datos. Siempre pasa por el controlador, que usa los servicios y los repositorios del modelo.

Cómo ejecutar el proyecto

Tener instalado Java (JDK) 17 o superior.

● Descargar el código desde el repositorio de GitHub.

● Abrir una terminal en la carpeta raíz del proyecto (donde se encuentra el archivo pom.xml) y ejecutar el siguiente comando:

En Windows (PowerShell/CMD): .\mvnw.cmd spring-boot:run

En Linux y macOS: ./mvnw spring-boot:run

(No hace falta instalar Maven, ya que el proyecto incluye el Maven Wrapper).

● Abrir en el navegador la dirección http://localhost:8080.

Las instrucciones completas, incluyendo cómo utilizar el carrito y cómo ver la base de datos H2, están en el archivo README.md del repositorio.

Conclusión

El patrón MVC nos permitió separar la aplicación en partes con tareas claras: el modelo maneja los datos, la vista muestra la información y el controlador conecta ambas. Gracias a esta separación pudimos agregar las órdenes de compra y sus detalles siguiendo la misma estructura que ya tenían los productos, sin cambiar el resto de la aplicación. También comprobamos que reutilizar componentes como Spring Data JPA y servicios compartidos reduce el código que hay que escribir y facilita su mantenimiento.